1. Problem Description :

The function accepts two positive integers ‘r’ and ‘unit’ and a positive integer array ‘arr’ of size ‘n’ as its argument ‘r’ represents the number of rats present in an area, ‘unit’ is the amount of food each rat consumes and each ith element of array ‘arr’ represents the amount of food present in ‘i+1’ house number, where 0 <= i

Note:

- Return -1 if the array is null
- Return 0 if the total amount of food from all houses is not sufficient for all the rats.
- Computed values lie within the integer range.

```
import java.util.*;
class Main
{
public static int solve (int r, int unit, int arr[], int n)
{
if (arr == null)
return -1;
int res = r * unit;
int sum = 0;
int count = 0;
for (int i = 0; i< n; i++)
{
sum = sum + arr[i];
count++;
if (sum >= res)
break;
}
if(sum<res)
return 0;
return count;
}
public static void main (String[]args)
{
Scanner sc = new Scanner (System.in);
int r = sc.nextInt ();
int unit = sc.nextInt ();
int n = sc.nextInt ();
int arr[] = new int[n];

for (int i = 0; i< n; i++)
```

```
arr[i] = sc.nextInt ();
System.out.println (solve (r, unit, arr, n));
}
}
```

- r: 7
- unit: 2
- n: 8
- arr: 2 8 3 5 7 4 1 2

4

Python:

```
def calculate(r,unit,arr,n): #2 8 3 5 7 4 1 2, r=7 unit=2 n=8
    if arr is None:
        return -1
    res=r*unit
    sum=0
    count=0
    for i in range(n):
        sum+=arr[i]
        count+=1
        if sum>=res:
            break
    if sum<res:
        return 0

    return count

if __name__=='__main__':
    r=int(input('Input number of rats '))
    unit=int(input('Input number of units of food each rat can consume '))
    n=int(input('Input number of houses'))
    arr=list(map(int,input('input food in each house ').split()))
    nHouses=calculate(r,unit,arr,n)
    print('%d houses can serve food for %d rats'%(nHouses,r))
```

2. Problem Description :

The Binary number system only uses two digits, 0 and 1 and number system can be called binary string. You are required to implement the following function:

int OperationsBinaryString(char* str);

The function accepts a string str as its argument. The string str consists of binary digits separated with an alphabet as follows:

- – A denotes AND operation
- – B denotes OR operation
- – C denotes XOR Operation

You are required to calculate the result of the string str, scanning the string to right taking one operation at a time, and return the same.

Note:

- No order of priorities of operations is required
- Length of str is odd
- If str is NULL or None (in case of Python), return -1

```
import java.util.*;
class Main
{
   public static int operationsBinaryString (String str)
   {
      if(str==null)
         return -1;
      int res = str.charAt (0) - '0';
      for (int i = 1; i<str.length ();i++)
      {
         char prev = str.charAt (i);
         if (prev == 'A')
            res = res & (str.charAt (i) - '0');
         else if (prev == 'B')
            res = res | (str.charAt (i) - '0');
         else
            res = res ^ (str.charAt (i) - '0');
i++;
      }
      return res;
   }

   public static void main (String[]args)
   {
      Scanner sc = new Scanner (System.in);
```

```
        String str =sc.next();
System.out.println (operationsBinaryString (str));
    }
}
```

1C0C1C1A0B1

1

Medium

The alphabets in str when expanded becomes “1 XOR 0 XOR 1 XOR 1 AND 0 OR 1”, result of the expression becomes 1, hence 1 is returned.

Python:

```
def operations_binary_string(s):
    if s is None:
        return -1

    res = ord(s[0]) - ord('0')
    i = 1

    while i < len(s):
        prev = s[i]
        i += 1
        if prev == 'A':
            res &= ord(s[i]) - ord('0')
        elif prev == 'B':
            res |= ord(s[i]) - ord('0')
        else:  # Assuming 'C' for XOR operation
            res ^= ord(s[i]) - ord('0')
        i += 1

    return res

# Main program
if _name_ == "_main_":
    str_input = input()
    print(operations_binary_string(str_input))
```

3. Problem Statement:

You are required to input the size of the matrix then the elements of matrix, then you have to divide the main matrix in two sub matrices (even and odd) in such a way that element at 0 index will be considered as even and element at 1st index will be considered as odd and so on. then you have sort the even and odd matrices in ascending order then print the sum of second largest number from both the matrices

```
import java.util.Scanner;

public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter size of array : ");
      int arrsize = sc.nextInt();
      int[] main = new int[arrsize];
      int[] even = new int[arrsize / 2 + arrsize % 2];
      int[] odd = new int[arrsize / 2];
      int evenCount = 0, oddCount = 0;

      System.out.println("Enter " + arrsize + " Elements");
      for (int i = 0; i < arrsize; i++) {
         main[i] = sc.nextInt();
         if (i % 2 == 0)
            even[evenCount++] = main[i];
         else
            odd[oddCount++] = main[i];
      }

      // Sort even and odd arrays
      sortArray(even, evenCount);
```

```
        sortArray(odd, oddCount);

        System.out.println("Even position index");
        printArray(even, evenCount);
        System.out.println("Odd position index");
        printArray(odd, oddCount);

        int evenSecondLargest = findSecondLargest(even,
evenCount);
        int oddSecondLargest = findSecondLargest(odd,
oddCount);

        System.out.println("Second Largest Element in Even List
is: " + evenSecondLargest);
        System.out.println("Second Largest Element in Odd List
is: " + oddSecondLargest);
        System.out.println("Sum Of Second Largest Element Of
Odd and Even List: " + (evenSecondLargest +
oddSecondLargest));
    }

    public static void sortArray(int[] arr, int size) {
        for (int i = 0; i < size - 1; i++) {
            for (int j = 0; j < size - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
```

```
    public static void printArray(int[] arr, int size) {
        for (int i = 0; i < size; i++) {
            System.out.println(arr[i]);
        }
    }

    public static int findSecondLargest(int[] arr, int size) {
        if (size < 2) {
            return -1; // Array too small to have a second largest
element
        }
        return arr[size - 2]; // Return second-to-last element of
sorted array
    }
}
```

Python:

```
def sort_array(arr):
    for i in range(len(arr)):
        for j in range(0, len(arr) - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

def print_array(arr):
    for item in arr:
        print(item)

def find_second_largest(arr):
    if len(arr) < 2:
```

```
        return -1  # Array too small to have a second largest
element
    return arr[-2]  # Return second-to-last element of sorted
array

# Main program
if _name_ == "_main_":
    arr_size = int(input("Enter size of array: "))
    main = []
    even = []
    odd = []

    print(f"Enter {arr_size} Elements")
    for i in range(arr_size):
        num = int(input())
        main.append(num)
        if i % 2 == 0:
            even.append(num)
        else:
            odd.append(num)

    # Sort even and odd arrays
    sort_array(even)
    sort_array(odd)

    print("Even position index")
    print_array(even)
    print("Odd position index")
    print_array(odd)

    even_second_largest = find_second_largest(even)
    odd_second_largest = find_second_largest(odd)
```

```
    print(f"Second Largest Element in Even List is: {even_second_largest}")
    print(f"Second Largest Element in Odd List is: {odd_second_largest}")
    print(f"Sum Of Second Largest Element Of Odd and Even List: {even_second_largest + odd_second_largest}")
```

4. Problem Statement :

A carry is a digit that is transferred to left if sum of digits exceeds 9 while adding two numbers from right-to-left one digit at a time

You are required to implement the following function, Int NumberOfCarries(int num1 , int num2);

The functions accepts two numbers ‘num1’ and ‘num2’ as its arguments. You are required to calculate and return the total number of carries generated while adding digits of two numbers ‘num1’ and ‘ num2’.

Assumption: num1, num2>=0

Example:

Sample Input

Num 1: 23

Num 2: 563

Sample Output

0

```
import java.util.*;
class Solution
```

```
{
   public static int numberOfCarries (int num1, int num2)
   {
      int count = 0;
      int temp1 = num1, temp2 = num2;
      int rem = 0;
      while (temp1 != 0 && temp2 != 0)
      {
         int d1 = temp1 % 10;
        int  d2 = temp2 % 10;
         if ((d1 + d2 + rem) > 9)
          {
            count++;
            int sum = d1 + d2 + rem;
            sum = sum / 10;
            rem = sum;
         }
         temp1 = temp1 / 10;
         temp2 = temp2 / 10;
      }
      return count;
   }
   public static void main (String[]args)
   {
      Scanner sc = new Scanner (System.in);
      int num1 = sc.nextInt ();
      int num2 = sc.nextInt ();

      System.out.println (numberOfCarries (num1, num2));
   }
}
```

Python:

```
def number_of_carries(num1, num2):
    count = 0
    temp1, temp2 = num1, num2
    rem = 0
    while temp1 != 0 and temp2 != 0:
        d1 = temp1 % 10
        d2 = temp2 % 10
        if (d1 + d2 + rem) > 9:
            count += 1
            sum_digits = d1 + d2 + rem
            rem = sum_digits // 10
        else:
            rem = 0
        temp1 = temp1 // 10
        temp2 = temp2 // 10

    # Check if there's any remaining carry after one number is
exhausted
    while temp1 != 0:
        d1 = temp1 % 10
        if (d1 + rem) > 9:
            count += 1
            sum_digits = d1 + rem
            rem = sum_digits // 10
        else:
            rem = 0
        temp1 = temp1 // 10

    while temp2 != 0:
        d2 = temp2 % 10
        if (d2 + rem) > 9:
            count += 1
```

```
        sum_digits = d2 + rem
        rem = sum_digits // 10
    else:
        rem = 0
    temp2 = temp2 // 10

  return count

# Main program
if _name_ == "_main_":
  num1 = int(input())
  num2 = int(input())
  print(number_of_carries(num1, num2))
```

5. Problem Statement:

A City Bus is a Ring Route Bus which runs in circular fashion. That is, Bus once starts at the Source Bus Stop, halts at each Bus Stop in its Route and at the end it reaches the Source Bus Stop again.

If there are n number of Stops and if the bus starts at Bus Stop 1, then after nth Bus Stop, the next stop in the Route will be Bus Stop number 1 always.

If there are n stops, there will be n paths.One path connects two stops. Distances (in meters) for all paths in Ring Route is given in array Path[] as given below:

Path = [1000, 1200, 750, 1500, 1000, 2200, 2000, 500]

Fare is determined based on the distance covered from source to destination stop as Distance between Input Source and Destination Stops can be measured by looking at values

in array Path[] and fare can be calculated as per following criteria:

- If d =1000 metres, then fare=5 INR
- (When calculating fare for others, the calculated fare containing any fraction value should be ceiled. For example, for distance 900m when fare initially calculated is 4.5 which must be ceiled to 5)

Path is circular in function. Value at each index indicates distance till current stop from the previous one. And each index position can be mapped with values at same index in BusStops [] array, which is a string array holding abbreviation of names for all stops as-

“Bus Stand” = ”BS”, “RlyStation” = “RS”, “GandhiRoad” = “GR”, “SVIMS” = “SVIMS”, “Alipiri” = “AL”, “KapilaTeertham” = “KT”, “LeelaMahal” = “LM”, “Srinivasam” = “SV”

Given, n=8, where n is number of total BusStops.

BusStops = [“BS”, ”RS”, ”GR”, ”SVIMS”, ”AL”, ”KT”, ”LM”,”SV”]

Path = [1000, 1200, 750, 1500, 1000, 2200, 2000, 500]

Write a code with function getFare(String Source, String Destination) which take Input as source and destination stops(in the format containing first two characters of the Name of the Bus Stop) and calculate and return travel fare.

```
def getFare(Source, Destination):
    BusStops = ["BS", "RS", "GR", "SVIMS", "AL", "KT", "LM",
"SV"]
```

```
    Path = [1000, 1200, 750, 1500, 1000, 2200, 2000, 500]

    source_idx = BusStops.index(Source)
    dest_idx = BusStops.index(Destination)

    # Calculate the total distance
    total_distance = 0
    if source_idx <= dest_idx:
        for i in range(source_idx, dest_idx):
            total_distance += Path[i]
    else:
        for i in range(source_idx, len(Path)):
            total_distance += Path[i]
        for i in range(0, dest_idx):
            total_distance += Path[i]

    fare = round(total_distance / 1000 + 0.5) * 5

    return fare

if __name__=='__main__':
    source = input(' Enter the source bus stop ')
    destination =input(' Enter the destination bus stop ')
    fare = getFare(source, destination)
    print(f"The fare from {source} to {destination} is {fare}
INR.")
```